



FMI 3.0 advanced topics

Presenters:

Christian Bertsch, Bosch Research
Claudio Gomes, Aarhus University
Maurizio Palmieri, University of Pisa

Special thanks to

Torsten Sommer, Dassault Systèmes

Online Materials:



Released under [Attribution-ShareAlike 4.0 International License](#)

<https://github.com/modelica/fmi-beginners-tutorial-2025/tree/main>

Content

FMI 3.0 news features

- New data types
- Binary Variables
- Co-Simulation with Events
- Intermediate variable access
- Clocks
- Adjoint derivatives

Layered Standards using FMI 3.0 features

- Layered Standard for Network Communication (FMI-LS-BUS)
- Layered Standard for structured Data (v1.0.0alpha) FMI-LS-STRUCT

Outlook

FMI 3.0: Main Improvements

- Event mode for Co-Simulation
- Intermediate variable update
- Clocks
- New variable types
- Array variables
- Terminals and icon
- FMI for Scheduled Execution (SE)
- Preparation for layered Standards

Performance

Accuracy

New Application

New Variable types

FMO 2.0 abstract types (e.g. “Real”)

```
<!-- Index for next variable = 1 -->
<ScalarVariable
  name="v"
  valueReference="352321536"
  causality="input">
  <Real
    declaredType="Modelica.Blocks.Interfaces.RealInput"
    unit="v"
    start="0.0"/>
  </ScalarVariable>
```

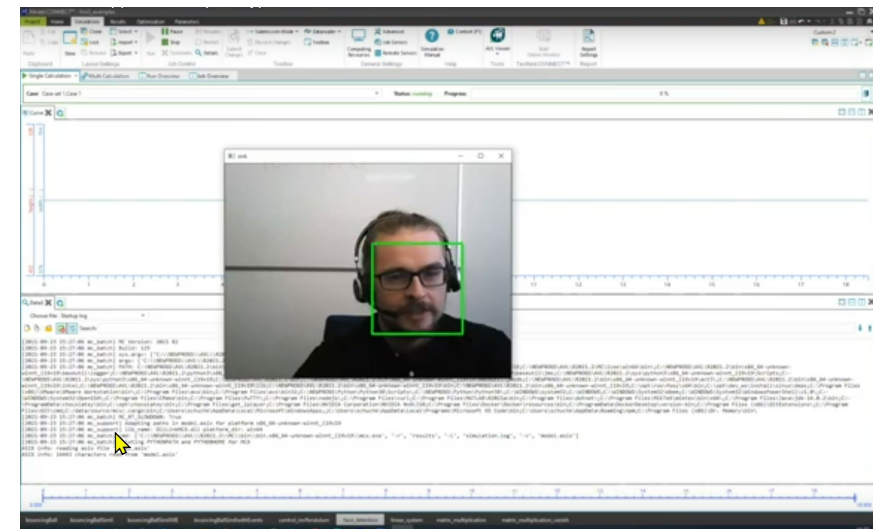
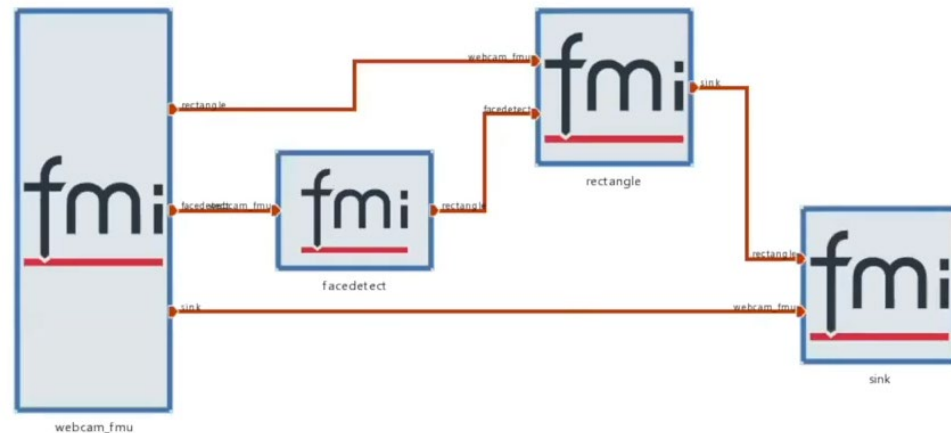
.fmi-beginners-tutorial-2025\part2\Drive.fmu

FMI 3.0 Concrete types (e.g. Float64)

```
<Float64
  name="v"
  valueReference="620756992"
  causality="input"
  intermediateUpdate="true"
  declaredType="Modelica.Blocks.Interfaces.RealInput"
  unit="v"
  start="0.0">
  <Annotations>
    <Annotation
      type="com.3ds.dymola">
      <DeclarationOrder
        order="4"/>
      </Annotation>
    </Annotations>
  </Float64>
```

Drive_FMI3.fmu

FMI 3.0 Binary Data



<https://www.youtube.com/watch?v=dhp3L6aMaAQ> FMI 3.0 Demo

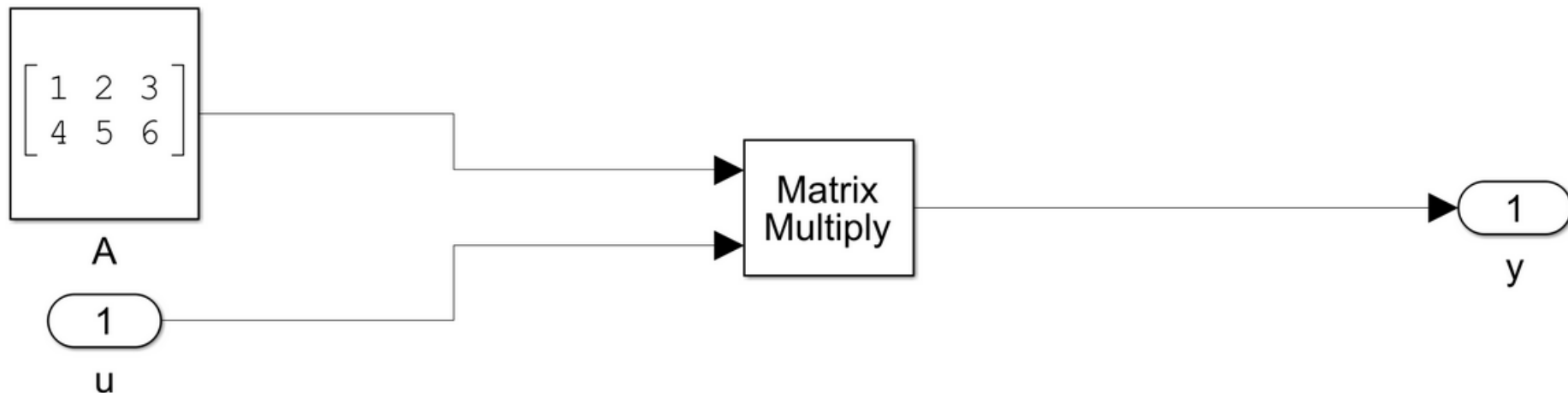
FMI 3.0 Array variables

FMI 2.0 supports arrays only via a naming convention, all variables are serialized

FMI 3.0 brings active array support

Array Variables

This notebook demonstrates how to access array variables efficiently with FMI 3.0 using the following model exported with [FMI Kit for Simulink](#) as an FMI 2.0 and FMI 3.0



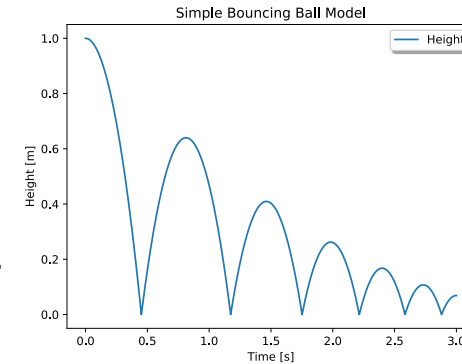
<https://github.com/chrbertsch/fmi3-features/blob/main>

 Open in Colab

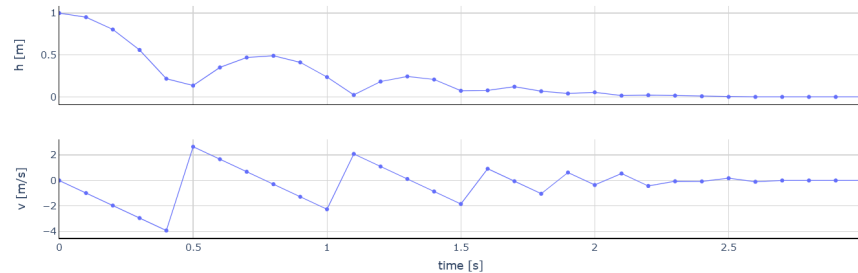
FMI 3.0 Co-Simulation with examples

Example: Bouncing ball

<https://github.com/t-sommer/fmi3-features/blob/main>



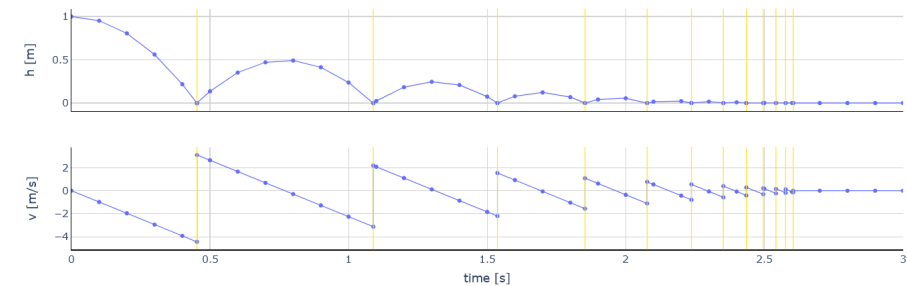
```
[1]: from fmpy import *
filename = 'resources/BouncingBall.fmu'
# simulate w/o events
result = simulate_fmu(filename, output_interval=0.1)
plot_result(result, markers=True)
```



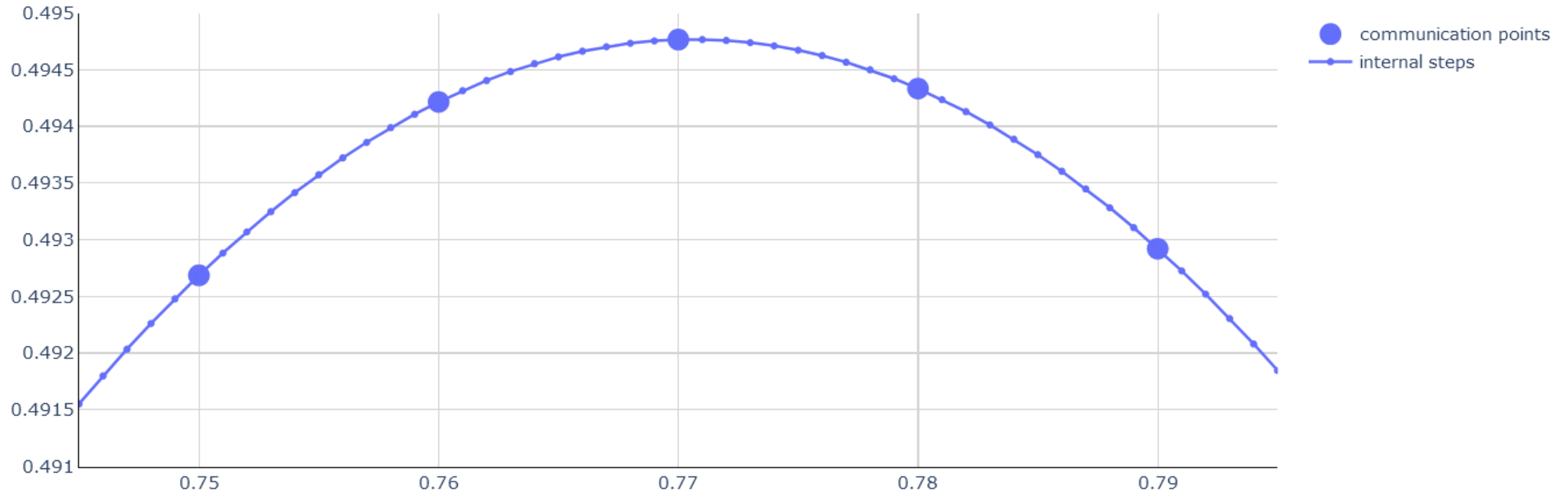
Simulation with FMI 2.0 CS

Exact timepoints where the ball hits the ground missed in Simulation of CS FMU with fixed communication timestep

```
[2]: # simulate w/ events
result = simulate_fmu(filename, output_interval=0.1, early_return_allowed=True, use_event_modes=True)
plot_result(result, events=True, markers=True)
```



Intermediate variable access

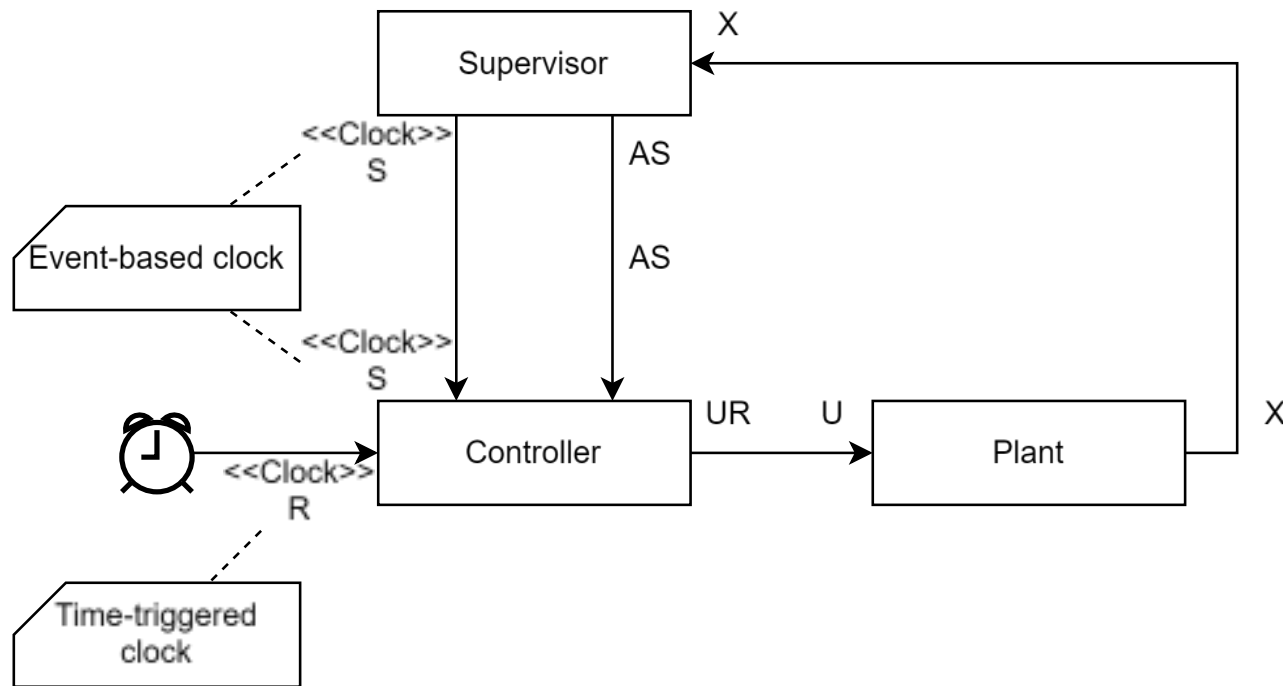


<https://github.com/chrbertsch/fmi3-features/blob/main>

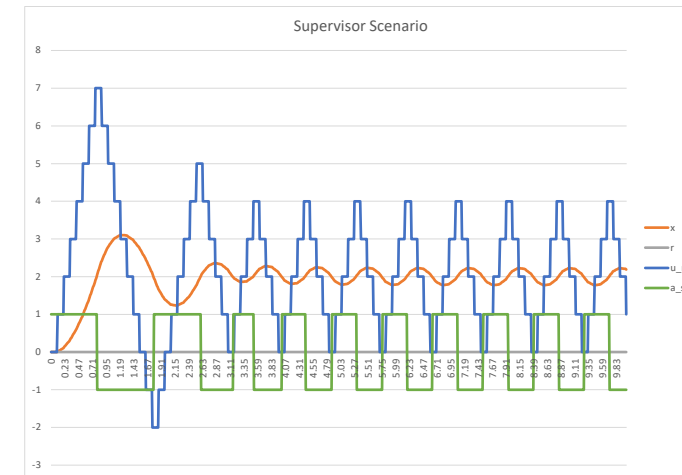
 Open in Colab

Clocks example

Clocks are used to synchronize events between different systems



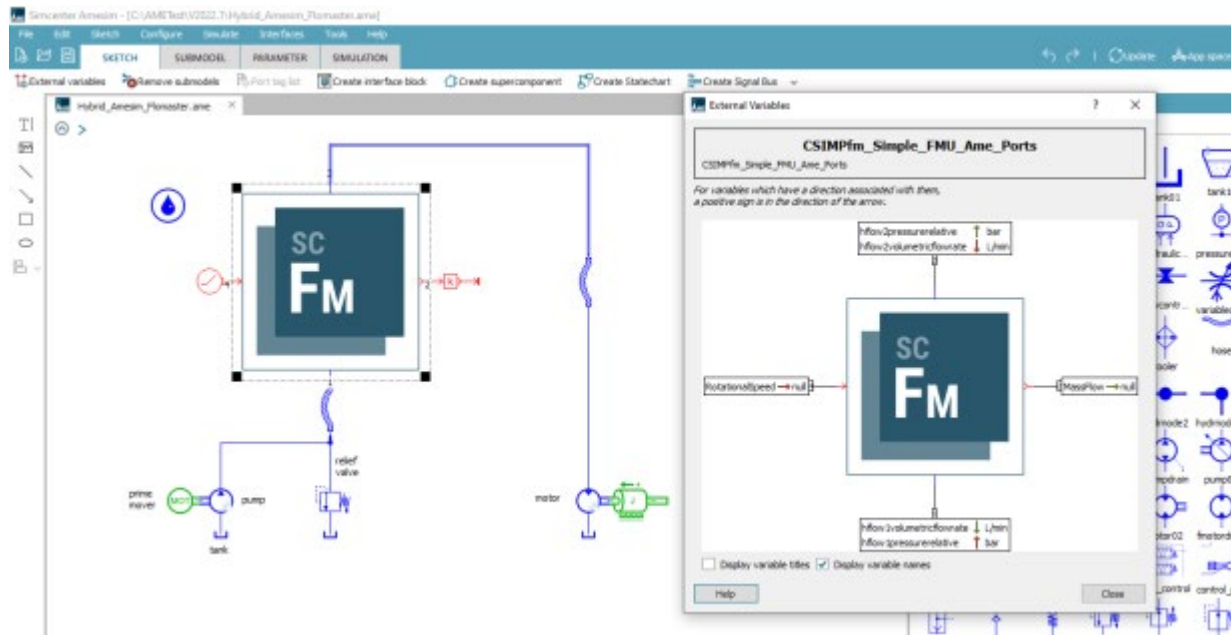
FMI 3.0 features used:
event-based, time-triggered Clocks



<https://github.com/clagms/synchronous-clock-fmus>

FMI Terminals

Grouping multiple signals into a new entity.



Source: <https://newsletter.modelica.org/2021-03/index>

Adjoint derivatives

Use FMI in optimization, # retrieve $v^T * J$ with a single call sensitivity =
`fmu_instance.getAdjointDerivative(unknowns, knowns, seed)`

FMI 3.0 features used:

https://github.com/t-sommer/fmi3-features/blob/main/adjoint_derivatives.ipynb

- directional, adjoint derivatives

```
# retrieve  $v^T * J$  with a single call sensitivity =  
fmu_instance.getAdjointDerivative(unknowns, knowns, seed)
```

E.g.. Implemented in



https://github.com/casadi/casadi/blob/main/docs/notebooks/fmu_demo.ipynb

Layered Standards

- Concept in FMI 3.0: Layered standards to allow standards extending FMI 3.0
Means: /extra directory, standardized annotations
- Layered Standards in development by the FMI Project:
 - **FMI-LS-XCP:** <https://github.com/modelica/fmi-ls-xcp> for XCP support
v1.0 released 12/2024
 - **FMI-LS-BUS:** <https://github.com/modelica/fmi-ls-bus> for network communication
v1.0 released 07/2025
Demo of interoperability at International ASAM Conference
 - **FMI-LS-STRUCT:** <https://github.com/modelica/fmi-ls-struct> for sampled maps and structuring of variables
v1.0.0-alpha.1 available
 - **FMI-LS-REF** <https://github.com/modelica/fmi-ls-ref> for the description of basic validation experiments and other files attached to an FMU

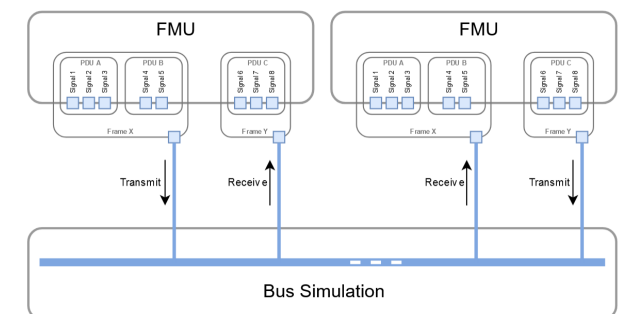
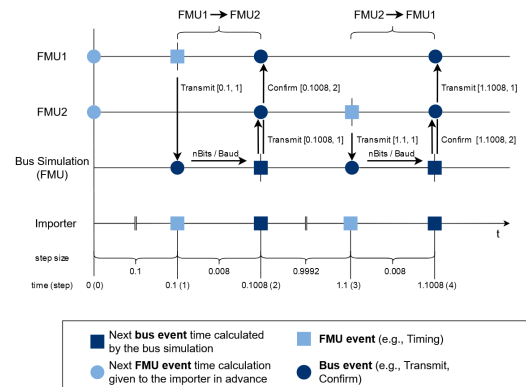


FMI-LS-BUS: Layered Standard Network Communication

Simulation of automotive systems requires network communication between vECUs.

Idea:

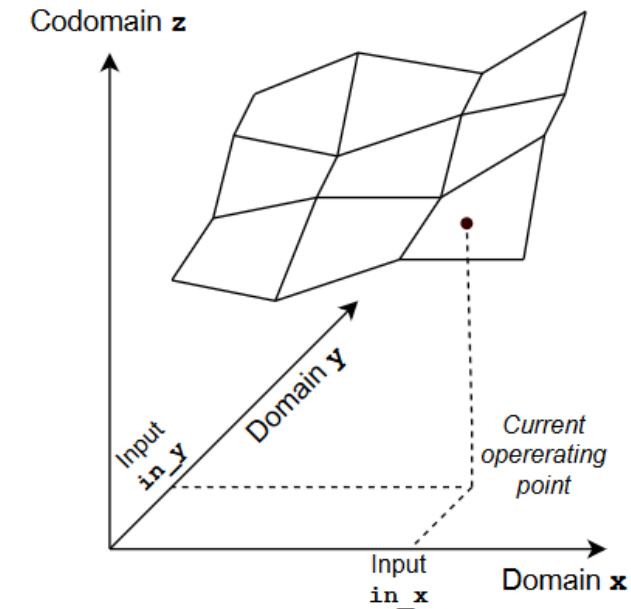
- Use **FMI 3.0 core standard features** to specify a common bus interface, using **Co-Simulation**, **Binary Variables**, **Clocks** and **Terminals**
- Support **automotive networks protocols** and others from **different industries** (CAN, LIN, FlexRay, Ethernet, ...)
- Using network description formats DBC, LDF,  **ASAM FIBEX**, ARXML
- Two variants:**
 - Physical abstraction layer ("high cut"):** Simply transport physical signal values between virtual ECUs by using unit-based and clocked variables
 - Network abstraction ("low cut"):** Detailed emulation of a specified bus system to realize virtualized bus driver implementations, including feedback from the physical drivers about transmission status or network node states



FMI LS-STRUCT: Layered Standard for Variable Structuring

Current scope:

- **Maps sampled on rectilinear grids**
using FMI 3.0 arrays by adding meta-data to define
 - Domains
 - Codomain
 - Optionally: Variables referencing current operating point, ...
 - Support for Modelica Standard Library CombiTables
 - Compatible with  **ASAM**-MCD2 MAPs, CURVEs, ...
- Maps sampled on an irregular grid ("Point Cloud")
- Using "FMI 3.0 Terminals" for grouping



Resources

- [FMI Webpage](#)

- [FMI tools list](#)
- [FMU validation](#)
- [Publications](#)

- [Reference FMUs](#)

- A set of hand-coded FMUs for development, testing and debugging of FMI.

- In case of questions we recommend to use [StackOverflow with tag „fmi“](#)

- You are welcome to join the (unofficial) [FMI LinkedIn Group](#)



Validate your FMUs

Whether you're exporting FMUs or troubleshooting a third party FMU - the following free tools help you to validate, test and debug your FMUs.



Thank you!

Any questions or comments?

Presenter Contacts

Christian.Bertsch Christian.Bertsch@de.bosch.com, contact@fmi-standard.org

Maurizio Palmieri maurizio.palmieri@ing.unipi.it

Cláudio Gomes claudio.gomes@ece.au.dk